

① Dining Philosophers Problem:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];
pthread_mutex_t mutex;

void *philosopher(void *num){
    int id = *(int *) num;
    while(1){
        pthread_mutex_lock(&mutex);
        printf("Philosopher %d status: THINKING\n",
            id);
        pthread_mutex_unlock(&mutex);
        sleep(1);
        pthread_mutex_lock(&mutex);
        printf("Philosopher %d status: WAITING\n",
            id);
        pthread_mutex_unlock(&mutex);
        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id+1)%N]);

        pthread_mutex_lock(&mutex);
        printf("Philosopher %d status: EATING\n", id);
        pthread_mutex_unlock(&mutex);
        sleep(2);
        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id+1)%N]);
    }
}

```

```

int main() {
    pthread_t tid[N];
    int i, ids[N];
    pthread_mutex_init(&mutex, NULL);
    for (i = 0; i < N; i++) {
        sem_init(&chopstick[i], 0, 1);
    }
    for (i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&tid[i], NULL, philosopher,
                      &ids[i]);
    }
    for (i = 0; i < N; i++) {
        pthread_join(tid[i], NULL);
    }
    return 0;
}

```

O/P: Philosopher 0 status: ^{THINK}~~WAITING~~

Philosopher 3 status: THINKING

Philosopher 2 status: THINKING

Philosopher 1 status: THINKING

Philosopher 4 status: THINKING

Philosopher 4 status: WAITING

Philosopher 1 status: WAITING

Philosopher 1 status: EATING

Philosopher 0 status: Waiting.

Philosopher 3 status: WAITING

Philosopher 4 status: EATING.

© Deadlock Detection:

→ #include <stdio.h>

#include <stdbool.h>

#define P 10

#define R 10

int allocation [P][R], request [P][R], available [R];
int n, m;

void DetectDeadlock() {

bool finish [P];

int work [R];

for (int i = 0; i < n; i++) finish [i] = false;

for (int i = 0; i < m; i++) work [i] = available [i];

bool deadlock = false;

for (int count = 0; count < n; count++) {

bool found = false;

for (int p = 0; p < n; p++) {

if (!finish [p]) {

bool canProceed = true;

for (int r = 0; r < m; r++) {

if (request [p][r] > work [r]) {

canProceed = false;

break;

}

}

}

if (!found) break;

}

for (int p = 0; p < n; p++) {

if (!finish [p]) {

deadlock = true;

printf ("Process %d is in deadlock.\n",

p);

}

}

```

if(!deadlock)
    printf("No deadlock detected\n");
}

```

```

int main(){
    printf("Enter no. of processes:");
    scanf("%d", &n);
    printf("Enter no. of resources:");
    scanf("%d", &m);
    printf("Enter allocation Matrix (%d x %d):", n, m);
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            scanf("%d", &allocation[i][j]);
        }
    }
    printf("Enter request Matrix (%d x %d):", n, m);
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            scanf("%d", &request[i][j]);
        }
    }
    printf("Enter Available resources (%d):", m);
    for(int i=0; i<m; i++){
        scanf("%d", &available[i]);
    }
    detectDeadlock();
    return 0;
}

```

O/p: Enter no. of processes: 3

Enter no. of resources: 3

Enter allocation matrix (3x3):

0	1	0
2	0	0
3	0	3

Enter request matrix (3x3):

0	0	1
1	0	0
0	0	2

Enter available resources (3): 1 0 0

Process 0 is in deadlock

Process 2 is in deadlock.